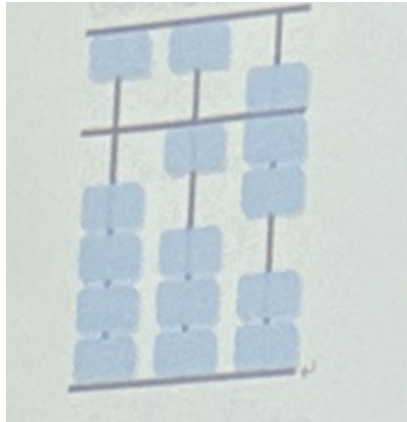


기출

2021

1. 세계최초의 전자식 컴퓨터는 (ABC)이다.
2. 컴퓨터의 범용성 개념을 처음 제시한 기기는 (ENIAC)이다.
3. 오늘날의 컴퓨터의 모델인 프로그램 내장형 컴퓨터를 처음 제안한 사람은 (폰 노이만)이다.
4. 1세대부터 4세대까지의 컴퓨터의 핵심 기본 소자를 순서대로 쓰시오
>> 전자석(0)→진공관(1)→트랜지스터(2)→집적회로(3)→초고밀도집적회로(4)
5. 컴퓨터 하드웨어의 기본 구성 요소 4개는 (중앙처리장치<CPU>, 기억장치, 입력장치, 출력장치)이다.
6. 튜링테스트와 CAPTCHA와의 관련성(같은 점, 다른 점)을 기술하라
>> 튜링 테스트는 기계가 인간과 얼마나 비슷하게 대화할 수 있는지를 기준으로 기계에 지능이 있는지를 판별하고자 하는 시험이다
>> CAPTCHA는 컴퓨터와 인간을 구분하기 위해 완전히 자동화된 튜링테스트이다
7. 2 Mega Bytes는 정확하게 계산하면 몇 바이트인가?
>> 2^{21} bytes
8. 기억장치의 접근 속도와 용량을 기준으로 5가지 종류를 접근 속도가 빠른 순서에서 느린 순서로 5가지 기술하시오
>> 레지스터>캐시메모리>주기억장치>보조기억장치
9. 중앙처리장치(CPU) 내부의 모듈 중에서, 내부 버스를 제외한 나머지 3개 모듈의 이름을 적으시오
>> 산술논리연산장치(ALU), 제어장치(CU), 레지스터
10. Multiprogramming과 Multitasking의 가장 큰 차이점
>> 멀티프로그래밍은 메인메모리에 여러 개의 프로그램이 동시에 떠있는 상황
>> 멀티태스킹은 다수의 작업을 운영체제 스케줄링에 의해 번갈아가면서 처리하는????
<https://oizys.tistory.com/9>
11. 10진수 +10 및 -10을 2의 보수로 각각 표현하시오(2진수 5비트)
>> +10 = 01010 / -10 = 10110
12. 주판을 이용하여 3자리 10진수 숫자를 표현하였다. 이 숫자를 양수라고 생각하고, 이 숫자를 10진수로 표현하시오. 그리고 이를 2진수와 16진수로 변환해서 쓰시오



>> 017(10진수) / 10001(2진수) / 11(16진수)

13. 2진수 10001010을 10진수로 변환하고자 한다. 동일한 2진수 패턴이라고 하더라도 이를 해석하는 상황에 따라 다른 숫자로 인식해야한다. 아래 각각의 경우에 해석되는 10진수를 적으시오

a. 부호 없는 정수

>> 138

b. 부호와 절댓값 표현

>> -10

c. 2의 보수 표현

>> -118

d. 부동 소수점 표현 (1개의 부호비트, 2개의 지수비트, 5개의 가수비트) (" -1.1111×2^3 "형식으로)

>> -1.01010×2^{-1}

14. 컴퓨터 내부에서는 2진수를 표현할 때 부호와 절댓값 표현은 사용하지 않고, 2의 보수 표현을 사용한다. 그 이유는 무엇인가?

>> 부호처리 없이 가산기만으로 음수 덧셈 가능 + 부호비트를 없애서 더 많은 범위의 값을 나타낼 수 있다.

15. 컴퓨터 프로그래밍 중에는 간혹 "unsigned int"타입의 변수를 사용하면 좋은 경우가 있는데, 그 이유를 2가지를 기술하시오.

>> 음수를 표현할 필요가 없는 경우, unsigned int를 사용하면 더 넓은 범위의 양수를 표현할 수 있다.

>> 연산이 편리하다

16. 프로세스 상태도에서 "실행 상태"에서 "대기 상태"로 전이할 수 있는 상황은 어떤 경우인가?

>> 프로세스가 입출력 또는 이벤트를 요청하고 이를 기다릴 때

17. 주기억장치관리기법으로 고정분할(fixed partition) 방식으로 사용하고 있다고 가정하자. 분할(partion) 할당 시에 worst fit 방법을 사용한다고 할 때, 아래와 같은 경우에 10K bytes 크기의 프로세스는 어느 분할에 할당되는가?

Partition 번호	Partition 크기(bytes)	상태
1	9K	빈 공간
2	12K	빈 공간
3	20K	빈 공간
4	35K	사용 중

>> 3번

18. byte 단위로 addressing하는 메인 메모리를 가지는 시스템이 있다. 이때 16-bit physical address를 가지며, 1페이지가 1Kbytes의 크기라고 가정하자

a. 페이지 내의 offset을 위해서는 주소가 몇 비트가 할당되어야하나?

>> 10 bit

>> 페이지가 2^{10} byte이므로 10개의 주소비트가 할당되어야한다.

b. 실제의 frame은 최대 몇 개인가?

>> 전체 physical address 16비트 중 offset에 10비트를 할당하였으므로 남은 6비트를 프레임에 할당할 수 있다

>> 따라서 2^6 개의 프레임을 가질 수 있다.

19. CPU 스케줄링 알고리즘 중, shortest-job first 알고리즘을 사용할 경우, 얻을 수 있는 장점과 단점은?

>> 장점 : 평균 대기 시간을 줄일 수 있다

>> 한계 : 그러나 각 프로세스가 얼마의 시간을 필요로 할지 모르는 한계가 있다. (걸리는 시간 미리 알 수 없음)

20. 페이징은 근본적으로 내부 단편화가 발생한다. '내부단편화의 크기(정도)'와 '페이지의 크기'의 상관 관계를 설명하라

>> 페이지의 크기보다 작은 크기의 프로세스가

2022

1. 컴퓨터를 구성하기 위해서 사용되었던 핵심 기본 소자들을 시대적 순서대로 기술하고, 각 소자들의 특징을 간단하게 기술하시오.

>> 전자석 → 진공관 → 트랜지스터 → 집적회로 → 초고밀도집적회로

>> 전자석/진공관 : 전력소모 큼, 발열 심함

>> 트랜지스터 : 전력 소비 안정적, 속도 빠름, 발열 적음

>> 집적회로 : 많은 트랜지스터를 하나의 칩에 집적함

>> 초고밀도집적회로 : 수천 개의 트랜지스터를 하나의 칩에 집적함

2. 컴퓨터 내에서 동작하는 각 구성 요소의 모든 동작들을 동기화하기 위해 사용하는 일정한 주기를 가지는 전자적 펄스를 (클록 펄스)라고 한다.

3. 컴퓨터의 범용성(general-purpose)이라는 의미를 ABC 컴퓨터와 ENIAC 컴퓨터를 예를 들어 설명하시오

>> 컴퓨터의 범용성이란 선형 방정식 연산만 가능했던 ABC 컴퓨터와는 ENIAC 컴퓨터처럼 여러 가지 용도로 사용할 수 있는 특징을 말한다.

4. 2 Mega Bytes는 정확하게 계산하면 몇 바이트인가?

>> 2^{21} byte

5. 출력값을 적으시오

```
signed char ch = 100;
cout << static_cast<int>(ch) << endl;
```

```
#include <iostream>
using namespace std;

int main() {
    signed char a = 200;
    signed char b = 100;
    cout << static_cast<int>(a) << endl;
    cout << static_cast<int>(b) << endl;
    cout << static_cast<int>(a + b);
    return 0;
}
```

>> 1) -56

>> 2) 100

>> 3) 44

6. 2진수 10001010을 10진수로 변환하고자 한다. 동일한 2진수 패턴이라고 하더라도 이를 해석하는 상황에 따라 다른 숫자로 인식해야한다. 아래 각각의 경우에 해석되는 10진수를 적으시오

a. 부호 없는 정수

>> 138

b. 부호와 절댓값 표현

>> -10

c. 2의 보수 표현

>> -118

d. 부동 소수점 표현 (1개의 부호비트, 3개의 지수비트, 4개의 가수비트) (분수형식으로 / 히든비트개념 사용)

>> -13/64

7. XOR 게이트 그림을 그리고, 이의 진리표를 작성하시

AND		$A \cdot B$	<table><tr><th colspan="2">입력</th><th>출력</th></tr><tr><th>A</th><th>B</th><th>A AND B</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	입력		출력	A	B	A AND B	0	0	0	0	1	0	1	0	0	1	1	1
입력		출력																			
A	B	A AND B																			
0	0	0																			
0	1	0																			
1	0	0																			
1	1	1																			
OR		$A + B$	<table><tr><th colspan="2">입력</th><th>출력</th></tr><tr><th>A</th><th>B</th><th>A OR B</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	입력		출력	A	B	A OR B	0	0	0	0	1	1	1	0	1	1	1	1
입력		출력																			
A	B	A OR B																			
0	0	0																			
0	1	1																			
1	0	1																			
1	1	1																			
NOT		$\overline{A}$	<table><tr><th>입력</th><th>출력</th></tr><tr><th>A</th><th>NOT A</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	입력	출력	A	NOT A	0	1	1	0										
입력	출력																				
A	NOT A																				
0	1																				
1	0																				
XOR		$A \oplus B$	<table><tr><th colspan="2">입력</th><th>출력</th></tr><tr><th>A</th><th>B</th><th>A XOR B</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	입력		출력	A	B	A XOR B	0	0	0	0	1	1	1	0	1	1	1	0
입력		출력																			
A	B	A XOR B																			
0	0	0																			
0	1	1																			
1	0	1																			
1	1	0																			
NAND		$\overline{A \cdot B}$	<table><tr><th colspan="2">입력</th><th>출력</th></tr><tr><th>A</th><th>B</th><th>A NAND B</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	입력		출력	A	B	A NAND B	0	0	1	0	1	1	1	0	1	1	1	0
입력		출력																			
A	B	A NAND B																			
0	0	1																			
0	1	1																			
1	0	1																			
1	1	0																			
NOR		$\overline{A + B}$	<table><tr><th colspan="2">입력</th><th>출력</th></tr><tr><th>A</th><th>B</th><th>A NOR B</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr></table>	입력		출력	A	B	A NOR B	0	0	1	0	1	0	1	0	0			
입력		출력																			
A	B	A NOR B																			
0	0	1																			
0	1	0																			
1	0	0																			

8. 아래의 그림의 박스 부분은 4비트 ripple-carry adder이다. 이 회로에 뺄셈 회로를 추가하시오.

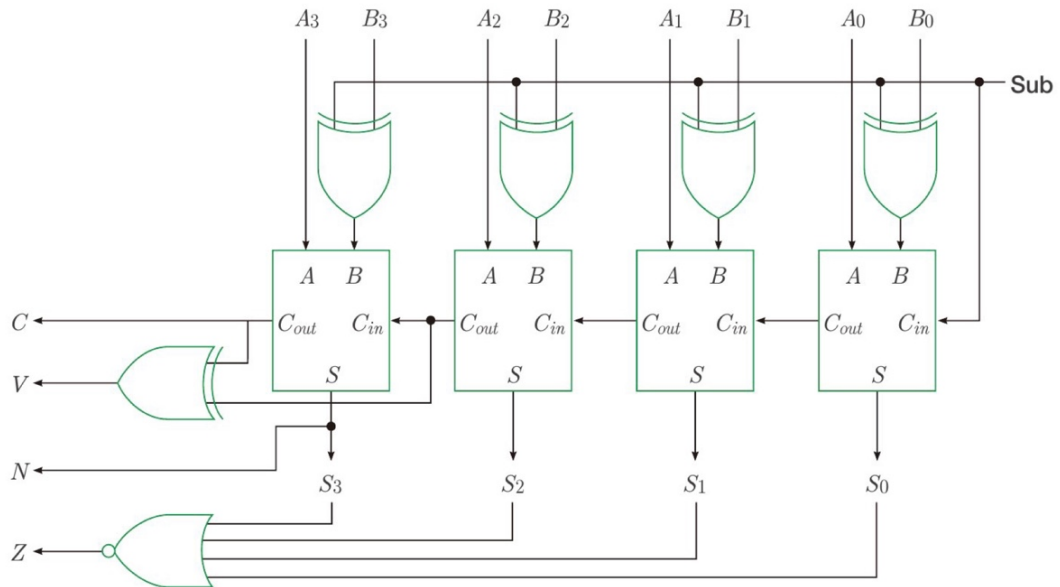


그림 2-29 4비트 전가산기의 논리회로

9. 캐시 메모리를 사용하면 메인 메모리 접근 속도를 향상시킬 수 있다고 한다. 그 근본 이유를 설명하시오.

>> 중앙처리장치와 메인메모리 사이의 속도차이로 인해 병목현상이 발생한다

>> 하지만 캐시 메모리를 사용하면 지역성의 원칙에 따라 사용할 가능성이 높은 데이터를 캐시메모리에 저장해두기 때문에 빠르게 데이터를 전달 할 수 있다

10. Multics, Unix Linux, Mac Os의 관련성을 간단하게 기술하시오

>> 이들은 모두 컴퓨터의 운영체제의 한 종류로써 자원관리, 자원보호, UI제공의 기능을 한다.

11. 지금 사용하는 컴퓨터가 Quad core인 단일 CPU라고 가정하자, 이 시스템에서 SIMD 병렬 처리를 하려고 한다. 어떻게 프로그래밍하면 될까? 배열(array) 데이터를 사용하는 상황에서 설명하시오

>> SIMD란 모든 CPU가 같은 명령어(프로그램)을 수행하지만 병렬적으로는 다른 데이터를 처리하는 구조를 의미한다.

>> 예를 들어, 배열 a와 b가 있고 이 두 배열의 합을 구하고자 하는 상황이라면

>> 덧셈이라는 하나의 명령(프로그램)을 각 배열에 저장된 서로 다른 데이터에 대해 병렬적으로 처리하는 것을 의미한다.